

RESEARCH ARTICLE

Anomaly Detection in Multi-Seasonal Time Series Data

ASHTON T. WILLIAMS, RYAN E. SPERL, AND SOON M. CHUNG^{ID}, (Life Member, IEEE)

Department of Computer Science and Engineering, Wright State University, Dayton, OH 45435, USA

Corresponding author: Soon M. Chung (soon.chung@wright.edu)

This work was supported in part by the Air Force Research Laboratory (AFRL)/Defense Associated Graduate Student Innovators (DAGSI) Research Fellowship.

ABSTRACT Most of today's time series data contain anomalies and multiple seasonalities, and accurate anomaly detection in these data is critical to almost any type of business. However, most mainstream forecasting models used for anomaly detection can only incorporate one or no seasonal component into their forecasts and cannot capture every known seasonal pattern in time series data. In this paper, we propose a new multi-seasonal forecasting model for anomaly detection in time series data that extends the popular Seasonal Autoregressive Integrated Moving Average (SARIMA) model. Our model, named multi-SARIMA, utilizes a time series dataset's multiple pre-determined seasonal trends to increase anomaly detection accuracy even more than the original SARIMA model. Our experimental results demonstrate the higher accuracy of multi-SARIMA when multiple seasonalities are present than most models with one or no seasonal component, although with more processing time.

INDEX TERMS Anomaly detection, moving average, multiple seasonalities, multi-SARIMA, time series data, SARIMA.

I. INTRODUCTION

Nowadays there are many data sources, such as sensors, producing time series data, which is a sequence of data points indexed in time order. These data points typically consist of successive measurements made from the same source over a fixed time interval and are used to track change over time [16]. Anomalies (i.e., outliers) are data points that significantly deviate from their expected value [4], and early detection of anomalies is important to mitigate these harmful effects, particularly in critical systems where failure can be catastrophic [3]. For example, a hospital can detect abnormal body signals of its patients and notify professionals before it's too late.

For anomaly detection in time series data, forecasting models are used to compare forecasted values to actual values to determine if a point is anomalous. While some deviation is expected when comparing a forecasted value to its real counterpart, if the predicted value deviates significantly from the actual value, then the data point is most likely an anomaly [2].

The associate editor coordinating the review of this manuscript and approving it for publication was Chao-Yang Chen^{ID}.

Some time series data contain a seasonality, which is a pattern that repeats at specific time intervals [1]. For example, CPU usage rate of a server may have a daily seasonal trend. The popular Seasonal Autoregressive Integrated Moving Average (SARIMA) forecasting model [18] can represent a seasonal trend in its forecasting of time series data. However, SARIMA can implement only one seasonal trend in its forecasting [1]. Allowing only one seasonal trend is a major limitation because some time series data contain more than one seasonality [7]. For example, New York City (NYC) taxi traffic data has both daily and weekly seasonal trends. Thus, utilizing all known seasonal effects in time series data can play an important role in data forecasting and anomaly detection [1].

In this paper, we propose a new multi-seasonal model, named multi-SARIMA, for anomaly detection in time series data that extends the SARIMA model by allowing multiple seasonal components. The multi-SARIMA utilizes a dataset's multiple pre-determined seasonal trends to increase anomaly detection accuracy. To compare with our multi-SARIMA, we also implemented other anomaly detection models, including Moving Average (MA), Seasonal Integrated Moving Average (SIMA), the original SARIMA,

Numenta's Hierarchical Temporal Memory (HTM) [9], [10], and another multi-seasonal model TBATS which stands for Trigonometric seasonality, Box-Cox transformation, ARMA errors, Trend and Seasonal components [8]. Additionally, we implemented the two-step approach proposed in [2] with our multi-SARIMA as the second step. The multi-SARIMA produced better anomaly detection results than the original SARIMA for every dataset we tested and, in most cases, outperformed HTM and TBATS.

This paper is organized as follows: Section II defines our anomaly labeling method and explains the existing anomaly detection models used in this paper. Section III explains our proposed multi-SARIMA model and the two-step approach we proposed in [2] with our multi-SARIMA model as the second step. Section IV describes the datasets used for testing and their properties, the Multiple Seasonal-Trend decomposition using Locally Estimated Scatterplot Smoothing (Loess) (MSTL) decomposition [7] we used to verify the seasonal trends in each dataset, and the differencing used on our datasets. Section V describes the implementation of different models, single-step and two-step test results, and comparisons based on the detection accuracy and runtime. Section VI contains our final thoughts with a conclusion and possible future research topics.

II. ANOMALY LABELING AND EXISTING DETECTION METHODS

A. ANOMALY LABELING

Time series anomaly detection models use a calculated numeric metric called an anomaly score to determine if a data point is an anomaly or not [4]. In our case, we determine the anomaly score using the error between the predicted value and the actual value. If the anomaly score exceeds the threshold, the data point is labeled as an anomaly [10].

In some cases, the threshold is fixed, however, a fixed threshold is not suitable in our case because the variance can change over time. The dynamic threshold must be calculated using sample metrics since we cannot assume the detector has access to the entire dataset as values are collected in real-time, such that only past values are available. For our dynamic threshold, we used the mean absolute deviation (MAD) calculated as:

$$MAD = \text{median}(|X_i - \text{median}(X)|) \quad (1)$$

where X is the portion of data values in a rolling sample window to limit the impact of older data values [2]. Unlike the mean and standard deviation, MAD is robust when anomalies are present in the data sample, and there is no distortion unless at least a half of the sample is composed of anomalies [2], [5]. We then multiply the threshold with a constant to adjust the sensitivity of our anomaly detection, resulting in our anomaly detection metric defined as:

$$\text{AnomalyDetected}_t = |\varepsilon_t| > |s * MAD| \quad (2)$$

where ε_t is the anomaly score measured at time t , and s is the sensitivity constant [2].

B. MOVING AVERAGE (MA)

Time series data is usually produced by a monitoring device and can range from spatial data in medical imaging to sequential data in network security [2], [12]. Let $X = \{X_1, X_2, \dots, X_t\}$ be a one-dimensional time series with evenly spaced discrete time where X_t is a value X at time t [2], [12]. Values that are before t are considered its lags, such that X_{t-i} is the value i steps back in the time series [2], [12]. The backshift operator B yields the lags in a time series and is defined as:

$$B^i X_t = X_{t-i} \quad (3)$$

for all $t > i$ [2], [12].

The moving average (MA) model is a simple and common approach to forecasting time series data. The moving average of order q , denoted as $MA(q)$, predicts the value X at time t as:

$$X_t = \mu + \varepsilon_t + \sum_{i=1}^q b_i B^i \varepsilon_t \quad (4)$$

for all $t > q$, where ε_t is the white noise error at time t , μ is the mean of the series, and $b = \{b_1, b_2, \dots, b_q\}$ is the q parameters for the model [2], [12].

C. SEASONAL INTEGRATED MOVING AVERAGE (SIMA)

The seasonal integrated moving average (SIMA) model is an extension of the MA model where one seasonal component is considered for the data forecast. SIMA denoted as $SIMA(d, q)_m$, forecasts using $MA(q)$ with a seasonally differenced time series. Differencing is used to eliminate trends that are apparent in a dataset to make the data stationary [2]. Differencing is done by replacing every value with the difference between itself and its first lag [2]. Let $\nabla X = \{X_1, X_2, \dots, X_t\}$ be a first-order differenced time series, such that

$$\nabla X_t = X_t - X_{t-1} = (1 - B) X_t \quad (5)$$

for all $t > 1$ [2]. The order of differencing can be represented by a symbol d , such that $\nabla^d X_t$ denotes the d th-order differenced time series. For example, when $d = 2$, it is the second-order differenced time series [2]. Therefore, a differenced time series can be expressed more generally as:

$$\nabla^d X_t = (1 - B)^d X_t \quad (6)$$

for all $t > d$ [2]. However, no amount of differencing will remove a seasonal trend from data. Seasonal trends can be eliminated by seasonal differencing, which differences against the previous season instead of the first lag [2]. Let $\nabla_m^d X$ be a seasonally differenced time series, where m is the period of the seasonal trend, then it is defined as:

$$\nabla_m^d X_t = (1 - B^m)^d X_t \quad (7)$$

for all $t > d * m$ [2].

D. SEASONAL AUTOREGRESSIVE INTEGRATED MOVING AVERAGE (SARIMA)

The seasonal autoregressive integrated moving average (SARIMA) denoted as SARIMA(p, d, q)_m is an extension of the autoregressive integrated moving average model denoted as ARIMA(p, d, q) by incorporating a seasonal component into its forecasting model. Both models are based on a combination of the autoregressive model (AR) and the moving average (MA) model. The autoregressive model predicts X using its most recent lags [2]. Let AR(p) be an autoregressive model of order p that predicts the value of X at time t as:

$$X_t = c + \varepsilon_t + \sum_{i=1}^p a_i B^i X_t \quad (8)$$

for all $t > p$, where $a = \{a_1, a_2, \dots, a_p\}$ is the p parameters for the model [2], [12]. Let ARMA(p, q) be an autoregressive moving average model (ARMA), where p represents the order of AR and q represents the order of MA, defined as:

$$X_t = c + \varepsilon_t + \sum_{i=1}^p a_i B^i X_t + \sum_{j=1}^q b_j B^j \varepsilon_t \quad (9)$$

for all $t > \max\{p, q\}$ [2], [12]. The ARIMA(p, d, q) model predicts X by modeling the differenced series $\nabla^d X$ with an ARMA(p, q) model [2]. The SARIMA(p, d, q)_m model predicts X by modeling the seasonally differenced series $\nabla_m^d X_t$ with an ARMA(p, q) model:

$$X_t = c + \varepsilon_t + \left(\sum_{i=1}^p a_i B^i \nabla_m^d X_t \right) + \left(\sum_{j=1}^q b_j B^j \varepsilon_t \right) \quad (10)$$

where $X_t = \nabla_m^d X_t + \sum_{i=0}^{d-1} \nabla_m^i B^m X_t$ [2]. Although the SARIMA model is one of the best and most common time series forecasting models, it is unable to incorporate more than one seasonal trend into its forecasting.

E. HIERARCHICAL TEMPORAL MEMORY (HTM)

Hierarchical Temporal Memory (HTM) is a neural network-based machine learning algorithm derived from neuroscience that models spatial and temporal patterns in streaming data [9]. HTM works by simulating how the neocortex works in the human brain [13]. It is versatile and tolerable to noisy data and can detect even the most subtle anomalies, resulting in a low false positive rate with most real anomalies detected [3].

The learning of HTM can be broken down into three main parts: The first part is the encoder and the Sparse Distributed Representations (SDRs) [3]. SDRs help explain how brains can make semantic generalizations [13]. SDRs are represented by vectors that contain thousands of bits, and the encoder gives the bits meaning by encoding them to represent the properties of a representation [13]. The encoded properties of two SDRs are compared, and if they have 1-bit in the same location, then they share some similarities [13]. The more 1-bits the two SDRs share, the more semantically similar the two representations are [13].

The second part is the spatial pooler. The spatial pooler is responsible for learning spatial patterns present in the data. It starts by taking in a fixed number of encoded SDR bits then assigns a layer containing columns [13]. Each column has a set of potential synapses, a connection to the previous layer representing a subset of the input bits [13]. Connections between the layers are then determined based on the comparison between performance values and a performance threshold [13]. The active synapses of each column are then determined based on how many connected columns exist [13]. As more data are collected, the spatial pooler determines how many connected synapses of each column overlap with the input SDR bits, and activates columns with the most overlap [3]. Only active columns update their connections, then the network boosts or hinders columns accordingly to prevent columns from being too dominant [2].

The third and final part of the HTM learning model is the temporal memory. Temporal memory does two things: learns the sequences of SDRs produced by the spatial pooler and makes predictions [13]. The temporal memory establishes connections between cells in the spatial pooler's columns, then learns the connections between cells that reside in the same layer [13]. An active cell forms connections to other cells that were just active. This way, the cells can predict when they will likely become active by referring to their current connections [13].

HTM also calculates its own anomaly score by measuring the deviation between its predicted input and the actual input [9]. The anomaly score at time t denoted as s_t , is given as:

$$s_t = 1 - \frac{\pi(X_{t-1}) * a(X_t)}{|a(X_t)|} \quad (11)$$

where $a(X_t)$ is the sparse encoded value of the input at time t , $|a(X_t)|$ is the total number of 1-bits in $a(X_t)$, and $\pi(X_{t-1})$ is the internal prediction of $a(X_t)$ [10]. The anomaly score will be 0 if the current input is perfectly predicted or 1 otherwise [9]. To increase the anomaly detection accuracy, a short-term average of the prediction errors is computed, then a threshold is applied to the Gaussian tail probability to determine if a data point is truly an anomaly [10]. This second step in determining the anomaly score is the compliment of the tail probability and is defined as the anomaly likelihood:

$$L_t = 1 - Q\left(\frac{\mu'_t - \mu_t}{\sigma_t}\right) \quad (12)$$

where $\mu'_t = \frac{\sum_{i=0}^{W'-1} s_{t-i}}{W'}$, μ_t is the mean of the sample of past anomaly scores, σ_t is the standard deviation of the sample of past anomaly scores, μ'_t is the short-term average, Q is the Gaussian tail probability function, and W' is a window for a short term moving average [9], [10].

F. TBATS

The Trigonometric seasonality, Box-Cox transformation, ARMA errors, Trend and Seasonal components model, denoted as TBATS, is a forecasting model for complex time series that can include multiple seasonal periods,

high-frequency seasonality, non-integer seasonality, and dual calendar effects [8]. TBATS is currently one of the best multi-seasonal time series forecasting models and is the most common [1]. It utilizes a framework that incorporates Box-Cox transformations, Fourier representations with time-varying coefficients, and ARMA error correction [8]. The TBATS model requires pre-specified seasonal periods that are then modeled by a trigonometric representation based on the Fourier series [1], [11].

TBATS is an extension of the Box-Cox transformation, ARMA errors, Trend, and Seasonal components (BATS) model, where the addition of trigonometric seasonality creates a more flexible parsimonious approach [8]. BATS, however, is an extension of exponential smoothing methods that combine its other components like Box-Cox transformations and ARMA errors to produce a better forecasting model [15]. The exponential smoothing in BATS utilizes the Holt-Winters method that handles time series with a trend and a single seasonality [8], [15]. The exponential smoothing works by having future values be weighted averages of past values [15]. The Box-Cox transformation in the model stabilizes the variance and mean over time, making the time series stationary. ARMA errors in the model are applied to the residuals to capture any leftover information [15]. The trend captures long-term changes in the mean. Lastly, the seasonal component captures a time series' periodical variation [15].

The BATS model was improved to forecast time series with multiple seasonal components with the addition of trigonometric seasonality as well as updated versions of some methods used in BATS to create the TBATS model [8], [15]. The trigonometric seasonality in TBATS represents each seasonal component in a time series as a trigonometric representation based on the Fourier series [8], [15]. This addition allows the model to fit multiple, larger, and non-integer seasonal components with less run-time than the original BATS model [15].

The BATS model can be represented as:

$$y_t^{(\lambda)} = l_{t-1} + \phi b_{t-1} + \sum_{i=1}^T s_{t-m_i}^{(i)} + d_t \quad (13)$$

$$\alpha l_t = l_{t-1} + \phi b_{t-1} + d_t \quad (14)$$

$$b_t = \phi b_{t-1} + \beta d_t \quad (15)$$

$$d_t = \sum_{i=1}^p \phi_i d_{t-1} + \sum_{i=1}^q \theta_i e_{t-i} + e_t \quad (16)$$

where $y_t^{(\lambda)}$ is the Box-Cox transformed time series at time t , $s_t^{(i)}$ is the i th seasonal component, l_t is the local level at time t , b_t is the trend with damping at time t , d_t is the ARIMA(p, q) process, e_t is white noise, ϕ and θ are the ARIMA(p, q) coefficients, ϕ is the trend damping, α and β are the smoothing, T is the amount of seasonalities, λ is the Box-Cox transformation, and m_i is the length of the i th seasonal period [8], [11]. The BATS model is then extended by adding the trigonometric seasonal model and is represented

as seasonal components based on the Fourier series:

$$s_t^{(i)} = \sum_{j=1}^{k_i} s_{j,t}^{(i)} \quad (17)$$

$$s_{j,t}^{(i)} = s_{j,t-1}^{(i)} \cos \omega_j^{(i)} + s_{j,t-1}^{*(i)} \sin \omega_j^{(i)} + \gamma_1^{(i)} d_t \quad (18)$$

$$s_{j,t}^{*(i)} = -s_{j,t-1}^{(i)} \sin \omega_j^{(i)} + s_{j,t-1}^{*(i)} \cos \omega_j^{(i)} + \gamma_2^{(i)} d_t \quad (19)$$

where $\gamma_1^{(i)}$ and $\gamma_2^{(i)}$ are the smoothing parameters, k_i is the amount of harmonics for the i th seasonal period, and $\omega_j^{(i)} = 2\pi j/m_i$ [8], [11].

III. PROPOSED DETECTION METHODS

A. MULTI-SARIMA

Most forecasting models today include at most one seasonal component, and this unnecessary restriction only hinders their potential. Allowing a model to take full advantage of every known seasonal pattern in a dataset gives more options and possibilities for it to perform better. Our proposed model, named multi-SARIMA, extends the original SARIMA(p, d, q) _{m} model and is denoted as SARIMA(p_1, d_1, q_1) _{m_1} $\times$ (p_2, d_2, q_2) _{m_2} . It predicts X by modeling the seasonal differenced series $\nabla_{m_2}^{d_2} X$ with two SARIMA(p, d, q) _{m} models:

$$\begin{aligned} \nabla_{m_2}^{d_2} X_t = & \left(\sum_{i=1}^{p_1} a_{1,i} B^{m_1 i} \right) \nabla_{m_2}^{d_2} X_t \\ & + \left(\sum_{i=1}^{p_2} a_{2,i} B^{m_2 i} \right) \nabla_{m_2}^{d_2} X_t \\ & - \left(\sum_{j=1}^{p_2} \sum_{i=1}^{p_1} a_{1,i} a_{2,j} B^{m_1 i + m_2 j} \right) \nabla_{m_2}^{d_2} X_t + \varepsilon_t \\ & + \left(\sum_{i=1}^{q_1} b_{1,i} B^{m_1 i} \right) \varepsilon_t + \left(\sum_{i=1}^{q_2} b_{2,i} B^{m_2 i} \right) \varepsilon_t \\ & + \left(\sum_{j=1}^{q_2} \sum_{i=1}^{q_1} b_{1,i} b_{2,j} B^{m_1 i + m_2 j} \right) \varepsilon_t \end{aligned} \quad (20)$$

where $X_t = \nabla_{m_2}^{d_2} X_t + \sum_{i=0}^{d_2-1} B^{m_2 i} \nabla_{m_2} X_t$, m_1 is the shorter seasonal period, m_2 is the longer seasonal period, d_2 is the order of differencing, a_1 is the p parameters for the shorter period, b_1 is the q parameters for the shorter period, q_1 is the seasonal MA order of the shorter period, p_1 is the seasonal AR order for the shorter period, a_2 is the p parameters for the longer period, b_2 is the q parameters for the longer period, q_2 is the seasonal MA order of the longer period, and p_2 is the seasonal AR order for the longer period. The multi-SARIMA equation was derived by extending the original SARIMA equation. The multi-SARIMA equation contains seasonal AR and MA terms for individual season lengths m_1 and m_2 followed by additional terms that account for the combination of the two seasonal trends, and the backshift operator being scaled by the season length. We also included additional factors to account for the nonseasonal trend (p, d, q) in the multi-SARIMA. From there, we distribute the factors and solve for the lone X_t to get the final equation depicted above. We concluded that we only need to difference using d_2 and m_2 since differencing over the longer seasonal trend captures both seasonalities and makes the data stationary. We set $d_1 = 0$ since we difference

the data and obtain a stationary version using d_2 , eliminating d_1 from appearing in the multi-SARIMA equation.

In our approach, the first model is trained on three iterations of the shorter seasonal trend, while the second model is trained on three iterations of the longer seasonal trend. From the first model, we obtain the seasonal and non-seasonal autoregressive and moving average parameters, the residuals, and the constant. From the second model, we obtain just the seasonal autoregressive and moving average parameters. During the prediction, we apply the values from both models to the multi-SARIMA equation to get the prediction X at time t .

We expect the multi-SARIMA model to perform well, compared to other models, when it is used with datasets that contain two meaningful seasonal trends. If a dataset doesn't have meaningful seasonal patterns, the multi-SARIMA model is not expected to perform better. If a dataset contains two seasonal trends, but they are insignificant, then the multi-SARIMA model is not guaranteed to perform better.

Better performance entails that the multi-SARIMA model has higher anomaly detection accuracy, meaning a higher true positive rate with a lower false positive rate. With this higher precision, however, we also expect the runtime of the multi-SARIMA to be somewhat longer than those of other models. This is because the multi-SARIMA model requires more fitting and learning than other models as it uses two different models and learns over two seasonal periods.

B. TWO-STEP APPROACH

The two-step approach for anomaly detection was initially proposed by us in [2]. The algorithm consists of a simpler model that can label data fast with less accuracy and a more complex model that can label data accurately but requires more time [2]. In the two-step approach, the first step does the initial labeling with the faster but less accurate model, then the second step verifies the first step's labels with the slower but more accurate model [2]. The first model must pick up as many true positives as possible, then the second step denies most of its false positives and verifies its true positives [2]. So, this combined approach is limited to the true positive rate of the first model but reduces the false positive rate [2]. In the worst case, the first model finds every data point anomalous, causing the second step to verify every data point in the dataset [2]. The runtime of the two-step is at best slightly slower than the first model and at worst slightly faster than the second model.

Although the two-step approach is not new by itself, using our multi-SARIMA as the second step in the two-step approach is. Since we expect our multi-SARIMA to perform better than other models, when it is used with datasets that contain two meaningful seasonal trends, we also expect our multi-SARIMA to perform well when it is used as the second step in the two-step approach. For the two-step approach, better results entail maintaining the true positive rate of first step model while significantly reducing its false positive rate.

TABLE 1. Overview of the datasets.

Datasets	Interval	Seasonality #1	Seasonality #2	Anomalies	Total Values
NYC Taxi	30 min	Daily	Weekly	5	10,320
Synthetic Dataset 3	1 hour	Daily	Weekly	5	8,664
HotGym	1 hour	Daily	Weekly	5	3,887

For our experiments, we used the MA and SIMA models as our first step to create the initial labeling, then verified the labels with SARIMA, our multi-SARIMA, and TBATS. We denote a combination of two models used in the two-step approach as 'first step + second step'. For example, a two-step approach that uses MA as the first step and SARIMA as the second step is denoted as MA + SARIMA.

IV. DATASETS

We evaluated all models on three different datasets. Two datasets are from the Numenta Anomaly Benchmark (NAB), a collection of labeled, univariate, real-world time series data [6]. The third dataset is a synthetic time series dataset we created using our data generation tool. Since we are focused on multi-seasonal anomaly detection in time series data, the three datasets contain two meaningful seasonal trends, numerous hand-labeled anomalies, and enough data points to train and test models on. A general summary of each dataset is given in Table 1. We used a smaller version of the HotGym dataset as one anomaly occurred within the first three weeks of the data causes training issues with some models.

A. MSTL SEASONAL DECOMPOSITION

Since our multi-SARIMA model utilizes two seasonal components, we should confirm that our test datasets contain two meaningful seasonal trends. For that purpose, we used Multiple Seasonal-Trend decomposition using Locally Estimated Scatterplot Smoothing (Loess) (MSTL) [7]. There are a few multi-seasonal time series decomposition methods available, including Facebook's Prophet, TBATS, and Seasonal-Trend Decomposition using Regression (STR); however, we chose MSTL because it produces the lowest root mean squared error, is robust to outliers, has the smallest execution time, and is easy to use as it requires minimal parameters [7], [14].

MSTL decomposes an additive time series into a trend component, given seasonal components, and a residual component [7], [14]. MSTL is an extension of the Seasonal-Trend decomposition using Loess (STL) model as STL is only able to decompose time series with one seasonal component [7], [14]. Loess is a scatterplot smoothing technique that fits a curve to a scatterplot to determine the degree of the polynomial [14]. STL applies Loess to various transformations of the given time series and then extracts the trend and one seasonal component [14]. MSTL extracts each known seasonal component in a time series using STL one by one [7], [14]. MSTL

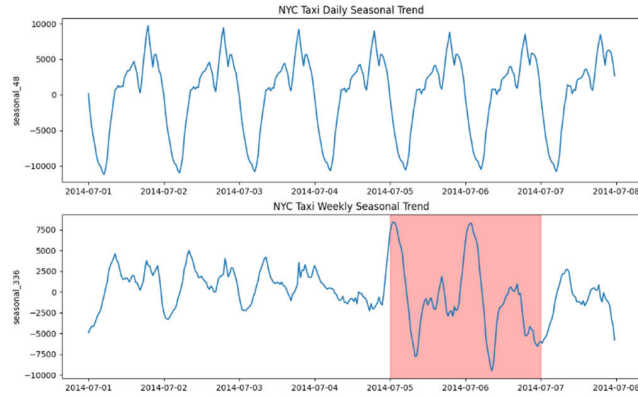


FIGURE 1. MSTL decomposition of the NYC Taxi dataset for one week of data. The top graph depicts the dataset's daily seasonal trend while the bottom graph depicts its weekly seasonal trend with the weekend highlighted in red.

first orders the given seasonal periods from shortest to longest to avoid shorter seasonal periods from being interlaced with the longer seasonal periods [7]. MSTL then applies STL iteratively on each identified seasonal period [7]. The MSTL additive decomposition of a time series can be defined as:

$$X_t = S_t^1 + S_t^2 + \dots + S_t^n + T_t + R_t \quad (21)$$

where $S_t^1, S_t^2, \dots, S_t^n$ denotes the seasonal components, T_t denotes the trend, and R_t denotes the remainder [7]. We used Python's statsmodels MSTL package on a Linux virtual machine to perform the MSTL decomposition on our datasets as depicted in Fig. 1–3.

Fig. 1 depicts one week of MSTL decomposition on the NYC taxi dataset, where the vertical axis represents the smoothing for the seasonal component given. The seasonality in the data follows a typical workweek and makes sense, considering that the original data represents taxi passengers in New York City. The daily trend is very low early in the morning, then has spikes before midday since everyone is trying to get to work, a dip around noon since no one is out and about, the highest spikes in the afternoon when everyone is heading home or traveling around the city, then ends with very a low dip late at night since everyone is at home. The weekly trend follows a typical workweek with the weekdays maintaining the same taxi usage pattern until the weekend showing a different pattern and higher spikes during later hours.

Fig. 2 depicts one week of MSTL decomposition on our Synthetic dataset. We generated our synthetic data to simulate a typical work schedule. The daily trend shows that the data values tend to be very low early in the morning, then has a spike before midday, a small dip around noon, another spike in the afternoon, and ends with very a low dip late at night. The weekly trend shows that the trend is consistent throughout the weekdays, then shifting to having lower values during the weekend.

Fig. 3 depicts one week of MSTL decomposition on the HotGym dataset. The seasonality in the data follows a typical

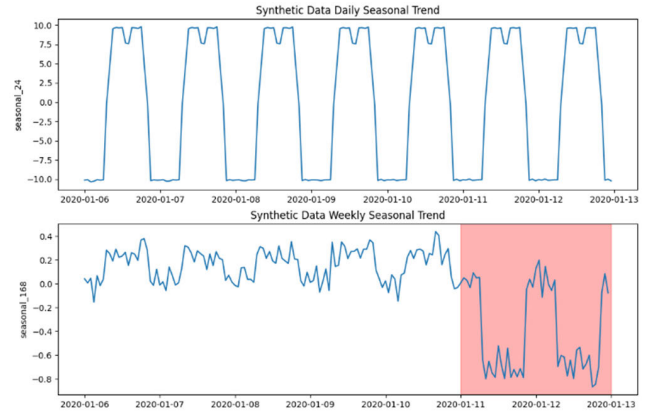


FIGURE 2. MSTL decomposition of the Synthetic Dataset for one week of data. The top graph depicts the dataset's daily seasonal trend while the bottom graph depicts its weekly seasonal trend with the weekend highlighted in red.

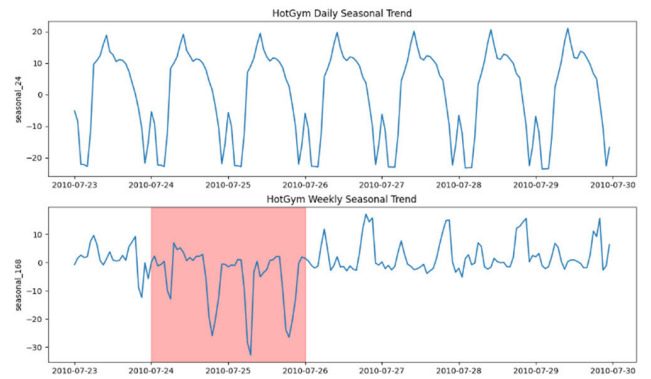


FIGURE 3. MSTL decomposition of the HotGym dataset for one week of data. The top graph depicts the dataset's daily seasonal trend while the bottom graph depicts its weekly seasonal trend with the weekend highlighted in red.

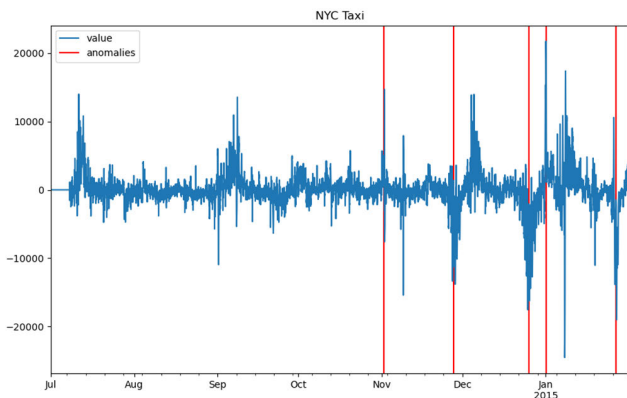
workweek and makes sense, considering that the original data represents a gym's energy consumption in Australia. The daily trend shows that the data values tend to have a small spike at midnight and then are very low early in the morning until midday where there is the highest spike during the hottest and busiest time of the day, then decreases back down for the rest of the day as the sun goes down and people go home. The weekly trend shows that throughout the weekdays, the trend seems to be somewhat consistent as people tend to visit the gym regularly during the week, until the weekend that has no high spikes but very low dips as not many people are going to the gym or it is closed during different hours.

B. DIFFERENCING

For the differencing of our test datasets in order to make them stationary, we decided to use seasonal differencing since they contain apparent seasonal trends, specifically daily and weekly seasonalities. So, we differenced using first-order seasonal differencing with a period of one week. This captures both the daily and weekly seasonal trends and produces

TABLE 2. Single-step experimental results.

Detector	NYC Taxi Dataset				Synthetic Dataset 3				HotGym Dataset			
	TP	FP	FN	Runtime (sec)	TP	FP	FN	Runtime (sec)	TP	FP	FN	Runtime (sec)
MA	2	654	3	2.002	2	3	3	2.084	4	195	1	0.722
SIMA	3	1587	2	3.444	5	105	0	2.349	2	591	3	0.965
SARIMA	2	1464	3	3.682	5	86	0	3.297	2	443	3	1.44
Multi-SARIMA	4	1425	1	1443.402	5	9	0	73.737	4	170	1	268.692
TBATS	3	1391	2	73.428	5	4	0	67.889	2	399	3	32.582
HTM	4	178	1	46.71	1	1	4	30.93	2	121	3	16.518

**FIGURE 4.** NYC Taxi dataset after first-order seasonal differencing with a period of 1 week. Anomalies are depicted by the red lines.

stationary data. Fig. 4 depicts our NYC Taxi datasets after the first-order seasonal differencing was applied. The beginning of the graph has a flat line because the first week has no prior data to difference against [2].

V. EXPERIMENTAL RESULTS

To properly compare our multi-SARIMA model, we used existing forecasting models MA, SIMA, SARIMA, TBATS, and HTM. MA, SIMA, SARIMA, TBATS, and our proposed multi-SARIMA were implemented in Python 3.8.5 on a Windows 10 computer with an Intel i7 8-core processor operating at 3.80 GHz, 16 GB of memory, and a 1 TB SSD. Numenta's HTM algorithm was implemented on the same machine, using Python 2.7. The optimal parameters for each model were determined by a grid search, and we compared the best performances of all models in this section. We used open-source python libraries provided by their authors for our implementations of HTM and TBATS. For MA, SIMA, and SARIMA we used Python's statsmodels package. For the two-step approach, we used MA and SIMA as the first step, and SARIMA, TBATS, and our proposed multi-SARIMA as the second step.

All models were trained on the first three weeks of the data, then evaluated on the remaining data. We made sure there were no anomalies present in the training portion of each

dataset. Since there are a very small number of anomalies within a large number of data points in each dataset, comparing performance based on accuracy percentage is ineffective as a model that never labels any data point as anomalous would achieve more than 90% accuracy [2]. Instead, we focus on which models produce the most *true positives* with the lowest number of *false positives*. This means, the best models would be able to label all anomalies correctly while not labeling other non-anomalous data points as anomalies. Table 2 shows the final single-step results of all models, and Table 3 shows the final two-step results, where TP is the number of true positives, FP is the number of false positives, and FN is the number of false negatives.

A. SINGLE-STEP EXPERIMENTAL RESULTS

For our single-step experimental results shown in Table 2, our multi-SARIMA model had the highest number of true positives for every dataset while maintaining fewer false positives than the SARIMA model for every dataset, although with longer runtime. Our multi-SARIMA had either the best or second-best results for every dataset.

The multi-SARIMA had the highest runtime compared to other models because the multi-SARIMA is the only model that combines the results from two models which train over the two seasonal periods of one day and one week, respectively. Since every other seasonal model but TBATS is limited to one seasonal trend, they are trained over the period of one day as that is their stronger seasonality. Training two models and having one training over a week required the extra time but produced better results. Specifically, the runtime of the multi-SARIMA on the NYC Taxi dataset was unexpectedly long. This is because the NYC Taxi dataset is the only dataset with a data point every 30 minutes instead of every hour, causing the 3-week training data to contain a large amount of data for the models to train on. The other multi-seasonal model, TBATS, was also slow and had the second longest runtime for every dataset. TBATS may be a more refined model, but it still requires more time since that is the nature of learning multiple seasonal patterns.

Our multi-SARIMA was the only model that achieved the same number of true positives as HTM for the NYC Taxi dataset and outperformed every model for the HotGym

TABLE 3. Two-step experimental results.

	NYC Taxi Dataset				Synthetic Dataset 3				HotGym Dataset			
Detector	TP	FP	FN	Runtime (sec)	TP	FP	FN	Runtime (sec)	TP	FP	FN	Runtime (sec)
MA + SARIMA	2	131	3	2.753	2	3	3	3.23	3	120	2	1.529
SIMA + SARIMA	3	1072	2	3.627	5	91	0	3.207	2	547	3	1.625
MA + Multi-SARIMA	2	93	3	787.783	2	1	3	69.95	3	53	2	94.329
SIMA + Multi-SARIMA	3	475	2	697.393	5	50	0	70.612	2	220	3	92.137
MA + TBATS	2	122	3	76.867	2	0	3	45.359	3	68	2	36.836
SIMA + TBATS	3	1156	2	79.023	5	4	0	47.08	2	306	3	35.939

dataset. The multi-SARIMA doubled the true positive rate of HTM and TBATS for the HotGym dataset while still maintaining the second lowest false positive rate among all models.

Expectedly, the two multi-seasonal models performed the best for the Synthetic dataset. Most models detected all five anomalies, but TBATS and multi-SARIMA did so with under ten false positives. HTM performed very poorly with this dataset, and we think that is because the dataset was created using randomness, throwing off the learning of HTM.

Notably, TBATS had either the same or higher true positive rate than the original SARIMA for every dataset, while maintaining a lower false positive rate.

B. TWO-STEP EXPERIMENTAL RESULTS

For our two-step experimental results shown in Table 3, all two-step algorithms, each of which uses a combination of two models, have less false positives than their standalone first step results shown in Table 2, except for MA + SARIMA for the Synthetic dataset which produced the same results as MA. This is because MA's false positives were already very low for that dataset. Also, most two-step algorithms have significantly less false positives than their standalone second step results shown in Table 2, but have less true positives because they are limited to the true positive rate of the first step.

Although with more processing time, the multi-SARIMA as the second step produced significantly less false positives than the original SARIMA as the second step for every dataset. The only case that produced less false positives than the multi-SARIMA is TBATS for the Synthetic dataset, which was expected as TBATS did better on that dataset. Also, the two-step approach using multi-SARIMA as the second step improved the runtime, compared to the standalone multi-SARIMA, as it worked on less data points. Notably, TBATS did better as the second step than the original SARIMA for every dataset, but worse than multi-SARIMA for two of the three datasets.

All two-step algorithms could not detect the four true positives that MA originally detected for the HotGym dataset. We believe this is because other models could not detect the

fourth anomaly detected by MA, causing them to label it as non-anomalous when they were used in the second step.

VI. CONCLUSION AND FUTURE TOPICS

When data contains repeated patterns such as seasonality, they can be learned and applied to a forecasting model to improve the accuracy of the model. Today, time series data containing multiple seasonalities are common in real-world applications [7]. However, most existing models for anomaly detection in time series data can include just one or no seasonal component, so they cannot capture every seasonal trend that appears in datasets.

Our multi-SARIMA model takes the original SARIMA model one step forward by including multiple seasonal components instead of just one. The multi-SARIMA produced better anomaly detection results than the original SARIMA for every dataset we tested and, in most cases, outperformed well-known HTM and TBATS. Also, we proved that our multi-SARIMA produces better results than SARIMA when used as the second step in the two-step approach we proposed in [2].

In addition to our multi-SARIMA model, we showed the anomaly detecting capability of an existing multi-seasonal forecasting model TBATS, which also outperformed SARIMA and HTM.

Different time series datasets have different characteristics, such that no one model could be the best for every case. However, our multi-SARIMA model showed very accurate detection performance on various datasets we used for evaluation and better overall results than other models.

In the future, we would like to incorporate some improvements to the multi-SARIMA model, including the runtime reduction, the ability to capture more than two seasonal trends, and a better way to choose optimal parameters. Moreover, we plan to compare the performance of our multi-SARIMA model with those of deep learning methods, such as Temporal Convolutional Networks (TCN) [17].

REFERENCES

- [1] T. Xie and J. Ding, "Forecasting with multiple seasonality," in *Proc. IEEE Int. Conf. Big Data*, Dec. 2020, pp. 240–245.
- [2] R. E. Sperl and S. M. Chung, "Two-step anomaly detection for time series data," in *Proc. Int. Conf. Data Softw. Eng. (ICoDSE)*, Nov. 2019.

- [3] Z. Hasani, "Robust anomaly detection algorithms for real-time big data: Comparison of algorithms," in *Proc. 6th Medit. Conf. Embedded Comput. (MECO)*, Jun. 2017.
- [4] C. C. Aggarwal, *Data Mining: The Textbook*. Cham, Switzerland: Springer, 2015.
- [5] J. Hochenbaum, O. S. Vallis, and A. Kejariwal, "Automatic anomaly detection in the cloud via statistical learning," 2017, *arXiv:1704.07706*.
- [6] A. Lavin and S. Ahmad, "Evaluating real-time anomaly detection algorithms—The Numenta anomaly benchmark," in *Proc. IEEE 14th Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2015, pp. 38–44.
- [7] K. Bandara, R. J. Hyndman, and C. Bergmeir, "MSTL: A seasonal-trend decomposition algorithm for time series with multiple seasonal patterns," 2021, *arXiv:2107.13462*.
- [8] A. M. De Livera, R. J. Hyndman, and R. D. Snyder, "Forecasting time series with complex seasonal patterns using exponential smoothing," *J. Amer. Stat. Assoc.*, vol. 106, no. 496, pp. 1513–1527, Dec. 2011.
- [9] S. Ahmad and S. Purdy, "Real-time anomaly detection for streaming analytics," 2016, *arXiv:1607.02480*.
- [10] S. Ahmad, A. Lavin, S. Purdy, and Z. Agha, "Unsupervised real-time anomaly detection for streaming data," *Neurocomputing*, vol. 262, pp. 134–147, Nov. 2017.
- [11] G. Skorupa, *Forecasting Time Series With Multiple Seasonalities Using TBATS in Python*. Accessed: Oct. 24, 2022. [Online]. Available: <https://medium.com/intive-developers/forecasting-time-series-with-multiple-seasonalities-using-tbats-in-python-398a00ac0e8a>
- [12] F. Orneholm, "Anomaly detection in seasonal ARIMA models," Dept. Math., Uppsala Univ., Uppsala, Sweden, Project Rep. 2019:18, 2019.
- [13] J. Hawkins et al. (2020). *Biological and Machine Intelligence. Release 0.4*. [Online]. Available: <https://numenta.com/resources/biological-and-machine-intelligence/>
- [14] K. Manani, *Multi-Seasonal Time Series Decomposition Using MSTL in Python*. Accessed: Dec. 5, 2022. [Online]. Available: <https://towardsdatascience.com/multi-seasonal-time-series-decomposition-using-mstl-in-python-136630e67530>
- [15] M. Peixeiro, *How to Forecast Time Series With Multiple Seasonalities*. Accessed: Dec. 6, 2022. [Online]. Available: <https://towardsdatascience.com/how-to-forecast-time-series-with-multiple-seasonalities-23c77152347e>
- [16] P. Dix, "What time series matters for metrics, real-time and sensor data?" InfluxData, San Francisco, CA, USA, to be published.
- [17] Y. He and J. Zhao, "Temporal convolutional networks for anomaly detection in time series," *J. Phys., Conf. Ser.*, vol. 1213, no. 4, Jun. 2019.
- [18] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Otexts, 2021.



ASHTON T. WILLIAMS received the B.S. and M.S. degrees in computer science from Wright State University, Dayton, OH, USA, in 2022 and 2023, respectively. He is currently a software engineer.

RYAN E. SPERL received the B.S. and M.S. degrees in computer science from Wright State University, Dayton, OH, USA, in 2019 and 2020, respectively. He is currently a software engineer.



SOON M. CHUNG (Life Member, IEEE) received the B.S. degree in electronic engineering from Seoul National University, South Korea, in 1979, the M.S. degree in electrical engineering from the Korea Advanced Institute of Science and Technology, South Korea, in 1981, and the Ph.D. degree in computer engineering from Syracuse University, Syracuse, NY, USA, in 1990. He is currently a Professor with the Department of Computer Science and Engineering, Wright State University, Dayton, OH, USA. His current research interests include database, data mining, text mining, information security, data grid, multimedia database, and parallel and distributed processing.

...